

Progetto di Laboratorio: Computer Vision per la "Smart Agriculture"

Scenario

Sei stato ingaggiato da una startup AgTech ("AgriFuture") che utilizza droni per monitorare lo stato di salute delle coltivazioni. Il tuo compito è sviluppare il cuore del sistema di visione artificiale: una rete neurale in grado di riconoscere diverse tipologie di colture o stati della pianta partendo da immagini aeree.

Il team di ingegneria ha deciso di utilizzare il **Transfer Learning** per risparmiare tempo e risorse computazionali, sfruttando reti già addestrate su grandi dataset.

Obiettivo

Creare uno script Python utilizzando **TensorFlow/Keras** che costruisca, compili e avvii l'addestramento di un modello di classificazione immagini basato sull'architettura **ResNet50**.

Specifiche Tecniche

Il tuo codice deve rispettare i seguenti requisiti obbligatori:

1. Architettura Base (Backbone):

- Non utilizzare *MobileNetV2*. Devi importare e utilizzare **ResNet50** pre-addestrata su ImageNet.
- La rete base deve essere importata **senza** il livello superiore (`include_top=False`).
- I pesi della rete base devono essere **congelati** (`trainable = False`) per non perdere la conoscenza pregressa.

2. Dataset e Classi:

- Il sistema deve essere configurato per classificare **5 classi** distinte (es. *Grano, Mais, Soia, Riso, Terreno vuoto*).
- Per questo esercizio, **simula i dati** (immagini random e label) utilizzando una funzione generatrice, senza caricare file esterni dal disco (lo script deve essere *standalone*).

3. Head Personalizzata (Classificatore):

- Aggiungi un livello di *GlobalAveragePooling2D*.
- Aggiungi un livello di *Dropout* (tasso 0.2 - 0.5) per la regolarizzazione.
- Aggiungi il livello *Dense* finale con attivazione **Softmax** adeguata al numero di classi.

4. Input Shape:

- Imposta la dimensione delle immagini a `(224, 224, 3)` (standard ottimale per ResNet).

Consegna Richiesta

Uno script Python (`.py` o cella Jupyter) che:

1. Definisca la funzione per generare dati dummy (simulati).
2. Definisca la funzione per costruire il modello (scaricamento ResNet50 + aggiunta strati).
3. Stampi il `model.summary()` per mostrare la differenza tra parametri totali e parametri addestrabili.
4. Esegua un addestramento di prova (2-5 epoche).

Suggerimenti per lo svolgimento

- **Importazione:** Attenzione a dove si trova ResNet50 in Keras. Solitamente il percorso è `tensorflow.keras.applications.resnet50`.
- **One-Hot Encoding:** Assicurati che la generazione delle etichette (`y`) nel generatore di dati corrisponda al numero di classi richiesto (5).
- **Dimensioni:** ResNet è una rete più profonda di MobileNetV2. Se il tuo computer è lento, riduci il `batch_size` o il numero di campioni simulati.

Criteri di Valutazione (Checklist)

- ☐ Il codice gira senza errori di sintassi?
- ☐ È stata importata `ResNet50` invece di `MobileNetV2` ?
- ☐ Il livello di output ha 5 neuroni?
- ☐ I pesi della base sono stati congelati correttamente?
- ☐ Viene usato l'ottimizzatore Adam e la loss `categorical_crossentropy` ?

(Bonus Facoltativo)

Implementa una sezione di hyperparameter tuning per ottenere un modello con migliori risultati.